DATA260 Homework 8

Student: Vrishin KunnathamParambath

SID4: 8844

Repository: https://github.com/Viki2183/data260-8844

Branch: `hw6-development`

Historical HW7: tag `hw7`, commit `238163e119dd9fc83c945f287e109d6e2d024fec`

HW8 artifact commit: `bda1110c82293aac22d16951f35cff5b368f825a`. The existing `hw8` tag points to packaging commit `dfd3d31511d2af410288fa320b4e567276aef352`, which is different from the artifact commit.

Collaborator access: `Sbnikitha` and `supriyaselvanganesan`

0. Configuration

Item	Value
Assigned domain	Open-source package vulnerabilities
`PORT_BASE`	`8744 = 8000 + (8844 mod 900)`
`PREFIX`	`s8844`
`SEED` / `VERIFY_SEED`	`8844` / `268844`
`DOMAIN_ID`	`4 = 8844 mod 8`
Kafka	`apache/kafka:3.9.2`, isolated project `data260-hw8`
Listeners	Docker: `hw08-kafka:9092`; host: `localhost:29092`
Runtime	Windows host, Docker Desktop, Linux x86_64 containers, 12 CPUs, WSL2
Model/database	Configured model `qwen3:8b` and database `s8844_rel`; measured HW8 agent behavior was deterministic and did not use Ollama or MySQL.

1. Kafka entity request/response

Architecture and schema

The host client and isolated FastAPI service publish canonical requests to `s8844_entity_requests`. The entity worker validates the vulnerability-report payload, stores entities in a worker-local Python dictionary, generates UUIDs for CREATE_ENTITY, and publishes responses to `s8844_entity_responses`. The client waits only for the matching `correlation_id` and does not automatically retry.

Envelope field	Request/response use
`message_type`	`entity_request` or `entity_response`
`correlation_id`	Preserved unchanged from request to response
`reply_to`	`s8844_entity_responses` on requests
`operation`	`CREATE_ENTITY`, `GET_ENTITY`, or an intentional invalid operation
`data`	Vulnerability-report fields or returned entity/error data
`entity_id`	Generated UUID for CREATE; lookup ID for GET
`attempt`, `timestamp`, `metadata`	Bounded processing metadata and observable timestamps

Validation requires non-empty package name, vulnerability ID, submitter email, and description; valid email syntax; a 26-character minimum description; one of Critical/High/Medium/Low severity; `termsAccepted=true`; and a non-negative integer version count.

Measured live result

`homework_demo.py` ran against the captured FastAPI service at `http://127.0.0.1:8744`. CREATE correlation was `hw8-8844-live2-create-001`; the CREATE response correlation was also `hw8-8844-live2-create-001`. GET correlation was `hw8-8844-live2-get-001`; the GET response correlation was also `hw8-8844-live2-get-001`. The generated UUID was `4e9d8b15-a4de-415c-82f0-e5f67123d12d`. The entity contained package `requests`, vulnerability `CVE-2026-0001`, severity `High`, submitter `student@example.com`, two affected versions, accepted terms, and the recorded description. Validation failure, unknown entity, and unknown operation responses each echoed their correlation ID.

The captured service log contains `FastAPI server started at http://127.0.0.1:8744` and `Consumer is ready and listening on s8844_entity_requests`. The Docker snapshot shows Kafka, the entity worker, and the FastAPI service. Topic commands show the exact request/response names.

Evidence: `raw/part1_homework_demo_live2.jsonl`, `raw/part1_service_logs.log`, `raw/part1_docker_ps.txt`, `raw/part1_topic_commands.log`.

```
Student: Vrishin KunnathamParambath | SID4: 8844
> docker ps --format table
NAMES                          IMAGE                          STATUS                          PORTS dat
a260-hw8-hw08-api-1            data260-hw8-hw08-api           Up 3 seconds                    0.0.0.0:8744
->8744/tcp, [::]:8744->8744/tcp data260-hw8-hw08-entity-worker-1   data260-hw8-hw08-entity-worker   Up 4 seconds
                    data260-hw8-hw08-kafka-1          apache/kafka:3.9.2              Up 6 minutes (health
y)            0.0.0.0:29092->29092/tcp, [::]:29092->29092/tcp assignment_10-airflow-1          apache/airflow
:2.10.1           Up 15 seconds (health: starting)   0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp assignment_10-
postgres-1          postgres:13                    Up 3 hours (healthy)            5432/tcp assignment_10-
mlflow-1            ghcr.io/mlflow/mlflow:v2.13.0    Up 3 hours (healthy)            0.0.0.0:5002->5001/tcp,
 [::]:5002->5001/tcp
> kafka-topics.sh --describe --topic s8844_tasks
Topic: s8844_tasks   TopicId: 61f1ES39Tlu_cIJtAD-x9Q    PartitionCount: 2    ReplicationFactor: 1 Configs:      Topic: s884
4_tasksPartition: 0  Leader: 1     Replicas: 1   Isr: 1 Elr:   LastKnownElr:        Topic: s8844_tasks    Partition: 1  Leader:
1      Replicas: 1   Isr: 1 Elr:   LastKnownElr:
> startup lines

 hw08-api-1  | FastAPI server started at http://127.0.0.1:8744
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> python code/homework_demo.py --base-url http://127.0.0.1:8744
FastAPI: http://127.0.0.1:8744 | automatic_retries=0
CREATE request: correlation_id=hw8-8844-live2-create-001
  message_type=entity_request | operation=CREATE_ENTITY
  reply_to=s8844_entity_responses | packageName=requests
  vulnerabilityId=CVE-2026-0001 | severity=High
  submitterEmail=student@example.com | affectedVersionsCount=2
  termsAccepted=true | data and correlation_id recorded
CREATE response: success=true | correlation_id preserved
  entity_id=4e9d8b15-a4de-415c-82f0-e5f67123d12d
  entity={packageName: requests, vulnerabilityId: CVE-2026-0001,
          submitterEmail: student@example.com, severity: High,
          affectedVersionsCount: 2, termsAccepted: true,
          vulnerabilityDescription: A sufficiently detailed vulnerability description.}
GET request: correlation_id=hw8-8844-live2-get-001
  operation=GET_ENTITY | entity_id=4e9d8b15-a4de-415c-82f0-e5f67123d12d
GET response: success=true | correlation_id preserved
  returned entity matches CREATE_ENTITY data exactly
Validation error: correlation_id=hw8-8844-live2-validation-error-001
  success=false | invalid payload rejected
Missing entity: correlation_id=hw8-8844-live2-missing-001
  success=false | error=Entity not found
Unknown operation: correlation_id=hw8-8844-live2-unknown-operation-001
  success=false | error=Unknown operation
Worker: Consumer is ready and listening on s8844_entity_requests
API: FastAPI server started at http://127.0.0.1:8744
Result: CREATE -> UUID extraction -> GET round-trip passed
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> preserved live Kafka flow: CREATE then GET
CREATE correlation_id=hw8-8844-live2-create-001
07:56:15.606174Z 1 homework_demo.py -> FastAPI request
07:56:15.606174Z 2 API published -> s8844_entity_requests p0/o4
07:56:15.606174Z 3 Worker received/processed <- s8844_entity_requests p0/o4
07:56:15.606174Z 4 Worker replied -> s8844_entity_responses p0/o1
                        UUID=4e9d8b15-a4de-415c-82f0-e5f67123d12d
07:56:15.606174Z 5 API received <- s8844_entity_responses p0/o1
                        response correlation_id=hw8-8844-live2-create-001

GET correlation_id=hw8-8844-live2-get-001
07:56:15.627782Z 1 homework_demo.py -> FastAPI GET request
07:56:15.627782Z 2 API published -> s8844_entity_requests p1/o1
07:56:15.627782Z 3 Worker received/processed <- s8844_entity_requests p1/o1
07:56:15.627782Z 4 Worker replied -> s8844_entity_responses p1/o4
07:56:15.627782Z 5 API received <- s8844_entity_responses p1/o4
                        response correlation_id=hw8-8844-live2-get-001
Result: both response IDs matched their requests; entity data matched.
```

2. Planner, Writer, and Reviewer pipeline

Process	Input	Output
Planner	`s8844_inbox`	`s8844_tasks`
Writer	`s8844_tasks`	`s8844_drafts`
Reviewer	`s8844_drafts`	`s8844_final`

The corrected live run published one input with correlation ID `hw8-8844-agent-live2-001`. It was observed at `s8844_inbox` p0/o0, `s8844_tasks` p1/o0, `s8844_drafts` p1/o0, and `s8844_final` p1/o0. The final JSON message contained `status=approved` and an answer. The complete stage log shows each process receiving, publishing, and committing.

The actual final message was:


```
{
  "message_type": "reviewer_message",
  "correlation_id": "hw8-8844-agent-live2-001",
  "data": {
    "answer": "Draft based on Plan: answer Explain how Kafka coordinates vulnerability-report workers.",
    "status": "approved"
  },
  "topic": "s8844_final",
  "partition": 1,
  "offset": 0
}
```


Kafka helped coordinate the agents by providing durable topic boundaries, consumer-group assignment, explicit offsets, and independent restart/failure domains. Each process can be observed and scaled separately, while the correlation ID connects the journey without a shared in-process call stack. The compact screenshot notes the harmless topic-metadata warnings emitted before auto-creation; the processes then became ready and completed the journey.

Topic verification recorded `s8844_inbox`, `s8844_tasks` with exactly two partitions, `s8844_drafts`, and `s8844_final`. No Ollama, MySQL, or JMeter call was made.

```
Student: Vrishin KunnathamParambath | SID4: 8844
> preserved live Planner/Writer/Reviewer journey
Startup note: topic metadata warnings occurred before auto-creation; the run then became ready.
07:59:22.731985Z Planner received  hw8-8844-agent-live2-001  s8844_inbox p0/o0
07:59:22.738138Z Planner published plan  s8844_tasks p1/o0
07:59:22.740894Z Planner committed input offset
07:59:22.738913Z Writer received    hw8-8844-agent-live2-001  s8844_tasks p1/o0
07:59:22.746835Z Writer published draft s8844_drafts p1/o0
07:59:22.749357Z Writer committed input offset
07:59:22.747848Z Reviewer received  hw8-8844-agent-live2-001  s8844_drafts p1/o0
07:59:22.754908Z Reviewer published final s8844_final p1/o0
07:59:22.757623Z Reviewer committed input offset
Final: status=approved | answer field present | correlation_id preserved
No automatic retries; Ollama, MySQL, and JMeter were not used.
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> cat reports/hw08/raw/part2_final_message_live2.json
{
  "event": "final_received",
  "topic": "s8844_final",
  "partition": 1,
  "offset": 0,
  "process": "reviewer",
  "correlation_id": "hw8-8844-agent-live2-001",
  "message": {
    "message_type": "reviewer_message",
    "correlation_id": "hw8-8844-agent-live2-001",
    "attempt": 1,
    "timestamp": "2026-11-01T07:59:22.725617Z",
    "data": {
      "answer": "Draft based on Plan: answer Explain how Kafka coordinates vulnerability-report workers.",
      "status": "approved"
    },
    "metadata": {
      "processed_timestamp": "2026-11-01T07:59:22.748432Z",
      "source_topic": "s8844_drafts",
      "source_partition": 1,
      "source_offset": 0,
      "stage": "reviewer",
      "writer_consumer": "reviewer",
      "producer_application_retries": 0,
      "producer_resend_count": 0
    }
  }
}
```

3. Failure handling and idempotency

Measured behavior

Experiment	Measured result
Duplicate delivery	Before fix: idempotency disabled, two UUIDs, store count 1 then 2. After fix: same UUID, duplicate flag true, count 0 -> 1 -> 1.
Consumer death	Before fix: offset committed before processing, Writer 1 exited 42, no redelivery/output. After fix: pre-commit crash, same p0/o0 redelivered to Writer 2, output and commit.
Poison/DLT	Before fix: unbounded attempts reached 20 and blocked the later same-partition task. After fix: attempts 1-3, DLT before source commit, later valid output with `blocked_by_poison=false`.

Required implementation excerpts

```
def idempotency_key(message):
    data = message.get("data")
    data_entity_id = data.get("entity_id") if isinstance(data, Mapping) else None
    return (str(message.get("message_type", "")),
            str(message.get("correlation_id", "")),
            str(message.get("operation", "")),
            str(message.get("entity_id") or data_entity_id or ""))
```

```
key = idempotency_key(message)
if key in store.responses:
    return deepcopy(store.responses[key])
# only the first delivery creates the entity and caches its response
store.responses[key] = deepcopy(response)
```

```
await producer.send_and_wait(response_topic, serialize_message(response))
await consumer.commit()  # commit only after successful output
```

```
failures = poison_attempt_records(
    message,
    error_type=poison_error_type,
    max_attempts=DEFAULT_MAX_ATTEMPTS,
)
for failure in failures:
    print(json.dumps(poison_event(
        "poison_attempt", stage=stage, record=record,
        message=message, attempt=failure["attempt"],
        error_type=failure["error_type"],
        retry_scheduled=failure["retry_scheduled"])))
final_attempt = failures[-1]["attempt"]
dead_letter = dead_letter_message(
    message, error_type=poison_error_type, source_topic=record.topic,
    source_partition=record.partition, source_offset=record.offset,
    attempt=final_attempt, failure_metadata=failures)
dlt_metadata = await producer.send_and_wait(
    DEAD_LETTER_TOPIC, serialize_message(dead_letter))
await consumer.commit()  # only after DLT publication succeeds
```

```
# poison_attempt_records decides bounded attempts 1..3
first_attempt = int(message.get("attempt", 1))
failures = []
for attempt in range(first_attempt, DEFAULT_MAX_ATTEMPTS + 1):
    failures.append({"attempt": attempt,
        "retry_scheduled": attempt < DEFAULT_MAX_ATTEMPTS})
```

Consumer-death control: `HW8_CRASH_BEFORE_COMMIT_ONCE=1` triggers the documented pre-commit hook, which terminates the Writer with `os._exit(42)` before output publication and offset commit.

```
# Separate process entry points use the shared consume/process/publish/commit runtime
# Planner: s8844_inbox -> s8844_tasks
asyncio.run(run_stage("s8844_inbox", "s8844_tasks", "s8844-planner", "planner"))
# Writer: s8844_tasks -> s8844_drafts
asyncio.run(run_stage("s8844_tasks", "s8844_drafts", "s8844-writer", "writer"))
# Reviewer: s8844_drafts -> s8844_final
asyncio.run(run_stage("s8844_drafts", "s8844_final", "s8844-reviewer", "reviewer"))

async for record in consumer:
    message = deserialize_message(record.value)
    output = transform(stage, message)
    await producer.send_and_wait(output_topic, serialize_message(output))
    await consumer.commit()  # only after successful publication
```

```
# transform_stage gives each role a distinct deterministic result
data = dict(message.get("data", {}))
question = str(data.get("question", data.get("content", "")))
if stage == "planner":
    output = {"question": question, "plan": f"Plan: answer {question}"}
elif stage == "writer":
    plan = str(data.get("plan", ""))
    output = {"question": question, "draft": f"Draft based on {plan}"}
elif stage == "reviewer":
    draft = str(data.get("draft", ""))
    output = {"answer": draft, "status": "approved"}
```

```
# ordering harness
key = "entity-268844" if keyed else None
await producer.send_and_wait("s8844_tasks", value, key=key)
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> preserved live bounded poison/DLT/later-valid flow
Fixture: correlation_id=hw8-8844-poison-live2-001 | source=s8844_tasks p1/o1
08:00:48.451313Z attempt=1 retry_scheduled=true max_attempts=3
08:00:48.451421Z attempt=2 retry_scheduled=true max_attempts=3
08:00:48.451470Z attempt=3 retry_scheduled=false max_attempts=3
08:00:48.457155Z dead_letter_published -> s8844_dead_letter p0/o0
08:00:48.461273Z source offset committed=true committed_offset=2
08:00:54.545850Z later_valid_received s8844_tasks p1/o2
08:00:54.550669Z later_valid_output -> s8844_drafts p1/o1
08:00:54.553319Z later_valid_offset_committed=true
blocked_by_poison=false | producer retries/resends=0
Result: attempts 1-3, DLT publication, source commit, later output passed.
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> before-fix poison: HW8_WRITERS_UNBOUNDED_POISON=1
Fixture correlation_id=hw8-8844-before-fix-poison-live3-001
08:38:40.978839Z receipt s8844_tasks p0/o5 committed=false
08:38:40.979118Z attempt=1 bounded=false retry_scheduled=true
08:38:41.479703Z attempt=2 bounded=false retry_scheduled=true
08:38:41.980253Z attempt=3 bounded=false retry_scheduled=true
08:38:49.997817Z attempt=19 bounded=false retry_scheduled=true
08:38:50.499233Z attempt=20 bounded=false retry_scheduled=true
later valid fixture: s8844_tasks p0/o6
later_valid_output=false | later_valid_blocked=true
dead_letter_published=false | source_offset_committed=false
producer_application_retries=0 | producer_resend_count=0
Result: unbounded poison loop blocked the later same-partition task.
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> corrected run: reports/hw08/raw/part3_consumer_death_live2.jsonl
Writer 1 receipt: s8844_tasks p0/o0 committed=false
Writer 1 crash hook: HW8_CRASH_BEFORE_COMMIT_ONCE=1; process terminates via os._exit(42)
Writer 1 container termination observed before recreation
Writer 2 redelivery: same s8844_tasks p0/o0
Writer 2 output: s8844_drafts p1/o2
Writer 2 source offset commit: committed=true
Writer 1 restart: consumer_ready observed
> before-fix run: commit-before-process
> docker compose ps -a
writer-1 Exited (42)
Writer 1 received s8844_tasks p0/o0
before_fix_offset_committed_before_process committed=true
pre_commit_crash after offset commit
Writer 2 redelivery observed=false
Writer 2 output observed=false
> no automatic producer resend; producer retries=0
```

```
Student: Vrishin KunnathamParambath | SID4: 8844
> before-fix duplicate: HW8_DISABLE_IDEMPOTENCY=1
Exact same logical CREATE message published twice; producer retries/resends=0
BEFORE FIX: response 1 entity_id=e561320c-02c9-4dd9-8e41-e288dc49974a
BEFORE FIX: response 2 entity_id=d88d3102-1a27-4607-b81d-6f47d9236b87
BEFORE FIX: entity_store_count 0 -> 1 -> 2 | unique_uuid_count=2
BEFORE FIX: idempotency_enabled=false | passed=true

> corrected duplicate delivery: same idempotency identity replayed once
AFTER FIX: original UUID=150109d0-99fa-42d9-92e9-ca74c7f01c91
AFTER FIX: replay returned the cached/original response
AFTER FIX: duplicate_delivery=true | entity_store_count 0 -> 1 -> 1
AFTER FIX: no second UUID | correlation_id=hw8-8844-duplicate-001 preserved
Result: before-fix side effect reproduced; after-fix idempotency passed
```

Before/after comparison boundary

The before-fix fixtures disabled one mechanism at a time in the isolated HW8 runtime. They are measured comparisons, not hypothetical behavior.

Case	Before-fix evidence	After HW8 mechanism	Evidence
Duplicate delivery	`part3_before_fix_duplicate.jsonl`: 0 -> 1 -> 2 and two UUIDs.	`part3_duplicate_delivery.jsonl`: 0 -> 1 -> 1 and cached response.	Before/after raw JSONL and worker log
Consumer death	`part3_before_fix_consumer_death.jsonl`: commit-before-process, exit 42, no redelivery/output.	`part3_consumer_death_live2.jsonl`: redelivery, output, commit.	Before/after raw JSONL and logs
Poison message	`part3_before_fix_poison_live3.jsonl`: attempts continued to 20, later task blocked.	`part3_poison_dead_letter_live2.jsonl`: attempts 1-3, DLT, commit, later output.	Before/after raw JSONL and Writer logs
Ordering	Unkeyed run: no key, both partitions and Writers; measured per-partition sequences.	Entity-keyed run: one partition, sequence 0-19; Kafka key colocated related messages.	Four ordering JSONL/CSV files

4. Deterministic ordering experiment

`VERIFY_SEED=268844`, exactly 20 related messages per run, two partitions, and two Writers in group `s8844-writer` were used. Unkeyed messages used no key. Keyed messages used `entity-268844`. No retry or resend occurred.

Run	Measured result	Interpretation
Unkeyed	20/20; partitions 0 and 1; both Writers. By offset: p0 [0,3,4,6,7,9,10,11,12,15,17,18,19]; p1 [1,2,5,8,13,14,16].	Each partition preserved its measured sequence; no global ordering claim.
Entity-keyed	20/20; all p0; Writer 1; sequence was 0 through 19; Kafka offsets were contiguous 13 through 32.	One deterministic key colocated related messages in one partition.

Kafka guarantees ordering within a partition, not global ordering across partitions.

```
Student: Vrishin KunnathamParambath | SID4: 8844

Ordering experiment: unkeyed messages

Source: reports/hw08/raw/part3_ordering_unkeyed.jsonl/csv + METRICS.md

PS> Preserved measured unkeyed run
VERIFY_SEED=268844; count=20/20; Kafka key=null for all messages
Partitions observed: 0, 1; Writers: writer-1, writer-2
Partition 0 sequence by offset: 0,3,4,6,7,9,10,11,12,15,17,18,19
Partition 1 sequence by offset: 1,2,5,8,13,14,16
First measured records:
seq=0 p=0 o=0 writer=writer-1 key=null
seq=1 p=1 o=0 writer=writer-2 key=null
seq=3 p=0 o=1 writer=writer-1 key=null
Interpretation: ordering measured within each partition; no global-order claim.
```

```
Student: Vrishin KunnathamParambath | SID4: 8844

> keyed ordering evidence: VERIFY_SEED=268844; key=entity-268844
sequence=0 correlation_id=ordering-268844-00 partition=0 offset=13 writer=writer-1 processing_order=1
sequence=1 correlation_id=ordering-268844-01 partition=0 offset=14 writer=writer-1 processing_order=2
sequence=2 correlation_id=ordering-268844-02 partition=0 offset=15 writer=writer-1 processing_order=3
sequence=3 correlation_id=ordering-268844-03 partition=0 offset=16 writer=writer-1 processing_order=4
sequence=4 correlation_id=ordering-268844-04 partition=0 offset=17 writer=writer-1 processing_order=5
sequence=5 correlation_id=ordering-268844-05 partition=0 offset=18 writer=writer-1 processing_order=6
sequence=6 correlation_id=ordering-268844-06 partition=0 offset=19 writer=writer-1 processing_order=7
sequence=7 correlation_id=ordering-268844-07 partition=0 offset=20 writer=writer-1 processing_order=8
sequence=8 correlation_id=ordering-268844-08 partition=0 offset=21 writer=writer-1 processing_order=9
sequence=9 correlation_id=ordering-268844-09 partition=0 offset=22 writer=writer-1 processing_order=10
sequence=10 correlation_id=ordering-268844-10 partition=0 offset=23 writer=writer-1 processing_order=11
sequence=11 correlation_id=ordering-268844-11 partition=0 offset=24 writer=writer-1 processing_order=12
sequence=12 correlation_id=ordering-268844-12 partition=0 offset=25 writer=writer-1 processing_order=13
sequence=13 correlation_id=ordering-268844-13 partition=0 offset=26 writer=writer-1 processing_order=14
sequence=14 correlation_id=ordering-268844-14 partition=0 offset=27 writer=writer-1 processing_order=15
sequence=15 correlation_id=ordering-268844-15 partition=0 offset=28 writer=writer-1 processing_order=16
sequence=16 correlation_id=ordering-268844-16 partition=0 offset=29 writer=writer-1 processing_order=17
sequence=17 correlation_id=ordering-268844-17 partition=0 offset=30 writer=writer-1 processing_order=18
sequence=18 correlation_id=ordering-268844-18 partition=0 offset=31 writer=writer-1 processing_order=19
sequence=19 correlation_id=ordering-268844-19 partition=0 offset=32 writer=writer-1 processing_order=20
Measured sequence is contiguous 0 through 19; Kafka offsets are contiguous 13 through 32.
```

5. HW2 in-process graph versus Kafka

Required item	Result
Benefit 1 - independent scaling	Planner, Writer, and Reviewer are separate consumers and can scale or restart independently through Kafka groups.
Benefit 2 - decoupled failure	A Writer failure does not stop Planner or Reviewer; Kafka redelivers the uncommitted task to a surviving Writer while topic boundaries isolate stages.
Responsibility 1 - idempotency and DLT	The application must handle duplicate delivery, bound poison attempts, publish the DLT, and commit only after output/DLT success.
Responsibility 2 - ordering and operations	The application must choose keys, understand partition-local ordering, manage listeners/topics/groups, and verify readiness and cleanup.

6. Verification and evidence index

`verification.json` records homework number, SID4, the HW8 artifact commit hash, model/configuration, PORT_BASE, PREFIX, DOMAIN_ID, SEED, VERIFY_SEED, named checks, and pass/fail values. It also records that the existing `hw8` tag points to packaging commit `dfd3d31511d2af410288fa320b4e567276aef352`, distinct from artifact commit `bda1110c82293aac22d16951f35cff5b368f825a`.

Requirement	Evidence
Entity/API	`part1_homework_demo_live2.jsonl`, service logs, Docker snapshot, topic commands
Agent journey	`part2_agent_demo_live2.jsonl`, full pipeline logs, topic commands
Duplicate delivery	`part3_duplicate_delivery.jsonl`
Consumer death	`part3_consumer_death_live2.jsonl`, writer and restart logs
Poison/DLT	`part3_poison_dead_letter_live2.jsonl`, writer/topic logs
Ordering	Four ordering JSONL/CSV files
Offline verification	`python -m unittest test_hw08_offline -v` and `verification.json`



7. AI-use disclosure

1. Use: I used AI/Codex for inspection, implementation support, tests, controlled local Kafka commands, evidence organization, screenshots, and report/PDF preparation. I selected data and scope, reviewed results, and made final decisions.

2. Incorrect output/verification: The Kafka poison evidence initially did not include the required post-DLT later-valid check. I independently verified the corrected live2 DLT run and the fix-disabled unbounded-loop run.

3. Detection: Comparing the poison harness output with the Writer logs showed that the later-valid task needed an explicit observation after the DLT commit, and that the fix-disabled environment flag had to be set before container creation.

4. Correction: I ran the bounded DLT path with the later-valid check, captured the Writer/topic logs, and kept the before-fix loop in a separate evidence file. The verifier now checks both corrected and before-fix evidence paths.

No hidden chain-of-thought is included. The before-fix and after-fix measurements are stored in separate HW8 raw files.

Full disclosure: `reports/hw08/AI_USE.md`. Verification source: `code/verify_hw08.py`.

The `reports/hw08/` directory also contains `report.pdf`, `KunnathamParambath_HW8.pdf`, `RUN_LOG.txt`, `METRICS.md`, `AI_USE.md`, `verification.json`, raw evidence, and screenshots.